Building Generative AI in Colab

A complete guide to setting up Groq-powered language models with LangChain for text generation and chat applications.

Made with GAMMA

What We Built in Colab

Installed Groq SDK

Set up the Python client library to connect with Groq's AI platform

API Key Authentication

Secured access using environment variables and credential management

Text Generation

Successfully generated responses using large language models

LangChain Integration

Built a foundational chat system using the LangChain framework

Made with GAMMA

Understanding API Keys

What Is an API Key?

An API Key is a unique secret code that serves as your digital passport to AI services. It authenticates your identity, authorizes access to specific resources, and tracks your usage for billing purposes.

Why It Matters

- Prevents unauthorized access to your account
- Keeps your AI usage secure and monitored
- Functions like a password specifically for APIs

Your Code Implementation

```
os.environ["GROQ_API_KEY"] = getpass(
    "Enter your api key here: "
)
```

This code securely prompts for your key and stores it in environment variables, keeping credentials out of your source code.

Made with GAMMA

Introducing Groq

Ultra-Fast Inference Platform

Groq specializes in lightning-fast LLM responses using purpose-built hardware optimized for AI workloads

Low-Latency Generation

Engineered specifically for minimal response times, making real-time AI applications possible

LLaMA Model Support

Provides access to powerful open-source models like LLaMA 3 and LLaMA 4 variants

Why We Chose Groq

Groq outperforms many competing providers in speed tests while offering an intuitive Python SDK. The platform supports the latest LLaMA models and provides reliable uptime for production applications.

Made with GAMMA

Large Language Models Explained

What Is an LLM?

Large Language Models are AI systems trained on massive datasets of text. They understand context, generate human-like responses, and can perform tasks like translation, summarization, and explanation.

Key Capabilities

- Understand natural language context
- Generate coherent, relevant responses
- Translate between languages
- Summarize complex documents
- Explain technical concepts

Models We Used

```
openai/gpt-oss-120b
llama-3.1-8b-instant
meta-llama/llama-4-scout-17b-16e-instruct
```

Each model offers different trade-offs between speed, capability, and token limits. The LLaMA series provides excellent performance with lower computational requirements.

Made with GAMMA

LangChain Framework

Application Framework

LangChain provides a comprehensive toolkit for building production-ready AI applications with modular components

Tool Integration

Connects LLMs with external tools, databases, APIs, and data sources for enhanced functionality

Memory & Agents

Supports conversation memory, autonomous agents, and Retrieval-Augmented Generation (RAG) patterns

Why LangChain?

LangChain simplifies chatbot development by handling complex logic, managing conversation state, and providing pre-built integrations. It's the bridge between raw LLM capabilities and production applications.

Made with GAMMA

Creating a Simple Chat with Groq

Your Implementation Code

```
completion = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[
        {"role": "user",
         "content": "Explain in 2 sentences about ML"}
    ],
    stream=True
)
```

This creates a chat completion request that streams responses in real-time, providing immediate feedback as the model generates text.

Expected Output

> "Machine learning (ML) is a subset of artificial intelligence that enables computers to learn from data without being explicitly programmed. It uses algorithms to identify patterns and make predictions on new data."

This demonstrates basic AI text generation where the LLM understands the query and provides a concise, accurate explanation.

Made with GAMMA

Building Chat with LangChain

Initialize Model

```
model = ChatGroq(
    model="llama-3.1-8b-instant",
    temperature=0.0
)
```

Prepare Messages

Structure system prompts, human messages, and conversation history in LangChain format

Get Response

```
model.invoke(messages)
```

Execute the chat and receive AI-generated responses

Example Output

```
"J'adore le programmation."
```

This shows the complete flow from system prompt to human message to AI response. The model maintains context and generates appropriate responses based on the conversation history.

Made with GAMMA

Complete AI Chat System Architecture

01

Install Required Libraries

Set up Groq SDK, LangChain, and dependencies in your Colab environment

02

Configure API Credentials

Store your Groq API key securely using environment variables

03

Initialize LLM Model

Create a ChatGroq instance with your chosen model and parameters

04

Structure Messages

Prepare conversation history with roles (system, user, assistant)

05

Generate Response

Call model.invoke() to get AI-generated text

06

Add Memory (Optional)

Implement conversation memory to maintain context across interactions

07

Integrate Tools (Optional)

Connect external APIs and data sources for enhanced capabilities

System Flow

User Input → LangChain Processing → **Groq API** → LLM Processing → **AI Response**

Made with GAMMA